

PROXMOX AUTOMATION SERIES · PHASE
2 FIELD GUIDE

Serial-Tagged Storage

LVM + XFS, Online

Four data disks declared in Terraform with stable serial identities, turned into PV, VG, LV, XFS and fstab mounts by one idempotent playbook, and grown any time with a single edit: no device-letter guessing, no manual resize commands, no reboot.

PVE 9.2 · TERRAFORM 1.15 · BPG/PROXMOX 0.111.1

ANSIBLE-CORE 2.20 · COMMUNITY.GENERAL · XFS

AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

DATA DISKS · SERIAL IDENTITY · RESIZE CHAIN

Contents

1. What Phase 2 Delivers	1
2. The Problem: Disk Guesswork and Manual Resizes	4
3. Design: The Serial Is the Disk's Identity	5
4. Terraform: Declaring the Four Disks	7
5. Apply: Disks Appear While the VM Runs	9
6. The Ansible Storage Playbook	11
7. First Run and Verification	14
8. The Resize Demo: One Edit, No Reboot	16
9. Troubleshooting	18
10. Where the Series Goes Next	20

1. What Phase 2 Delivers

This guide is the second module of the Proxmox Automation Series. Phase 1 built the provisioning pipeline: one Terraform command cloned a golden template, applied cloud-init networking, and handed the VM to Ansible. Phase 2 adds the layer that turns that cloned VM into something that looks like a production database server: serial-tagged data disks, LVM volume management, XFS filesystems, and an online resize chain that never requires a reboot.

The starting point is exactly where Phase 1 ended. VM 200 is running on pve-a with its final IP, the QEMU guest agent is installed and answers to `qm guest cmd 200 ping`, and the whole repository is under version control with its first commit. The provisioning layer is proven; what the VM still lacks is a storage layout. Right now it owns a single OS disk, which is fine for a test workload and wrong for anything that resembles real life.

In production DB-server practice the operating system gets one disk and the workload gets its own: a disk for `/var`, one for logs, one for data, one for backups, each on its own LUN or controller so that I/O loads stay separated and each volume can grow on its own schedule. Phase 2 reproduces that layout declaratively: four data disks declared in Terraform with stable serial identities, an idempotent Ansible playbook that turns each disk into a mounted XFS filesystem, and a resize procedure that is one `tfvars` edit, one `apply` and one playbook re-run long.

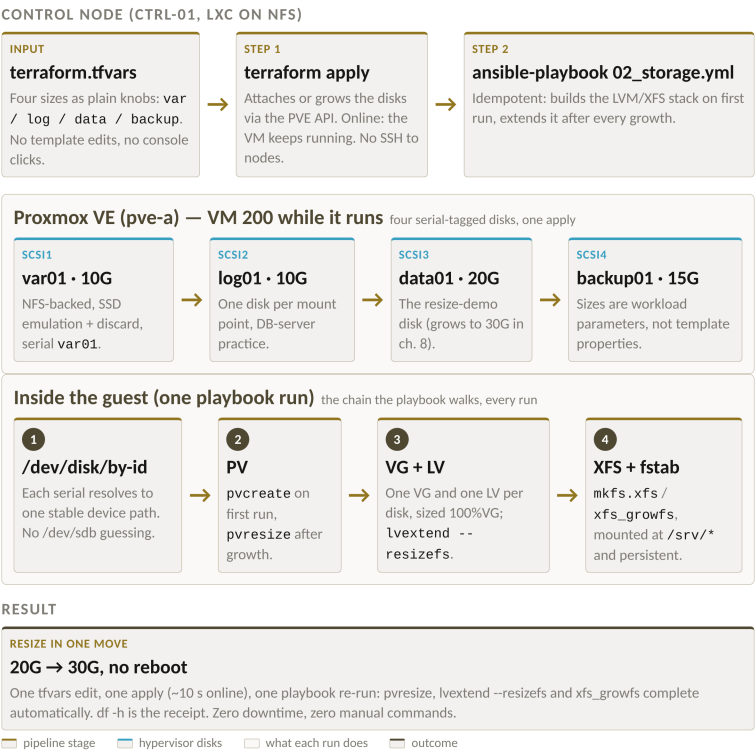


Figure 1. The Phase 2 storage pipeline: declare, attach online, build the LVM/XFS stack, grow at will.

What changed in the repository

Phase 2 touches three places and adds two files. Nothing from Phase 1 is removed, and because the new Terraform variables all have defaults, an existing `terraform.tfvars` keeps working without a single edit. The table below is the complete diff inventory; everything else in this guide explains why these lines look the way they do.

File	Change	Purpose
terraform/vm.tf	modified	Four new disk blocks (scsi1 to scsi4) with serial tags; big comment block documenting the identity scheme
terraform/variables.tf	modified	Four new size variables: var, log, data and backup disk sizes in GB, all with defaults
terraform/terraform.tfvars.example	modified	The new knobs, commented with their serial, interface and mount point
ansible/requirements.yml	new	Collection dependencies: community.general and ansible.posix
ansible/playbooks/02_storage.yml	new	The storage playbook: serial resolution, PV, VG, LV, XFS, fstab, receipts

Table 1. The complete Phase 2 diff: two files added, three modified.

Scope and boundaries

As in Phase 1, the scope is deliberately narrow so that every moving part can be verified in isolation. The storage playbook does one thing: it converges the disks that Terraform declares into mounted XFS filesystems. It does not migrate existing data, it does not manage the OS disk, and it does not attempt to shrink anything, because neither the provider nor XFS allows that. Those boundaries are documented below and revisited in the troubleshooting chapter.

In scope for Phase 2	Deferred to later phases
Four serial-tagged data disks on the workload VM	Adding disks to the golden template itself (template stays lean)
PV, VG, LV, XFS and fstab managed by one idempotent playbook	Migrating live directories such as /var onto the new disks (rsync dance)
Online growth: tfvars edit, apply, playbook re-run	Shrinking (impossible by design: provider hard error, XFS limitation)
Verification receipts: lsblk, df, serial mapping	Monitoring of filesystem usage (Zabbix arrives in Phase 3)
Community collections as a managed dependency	Dynamic inventory driven by Terraform state (Phase 4)

Table 2. What this guide builds, and what it intentionally leaves for later.

2. The Problem: Disk Guesswork and Manual Resizes

The requirements that shaped this phase came from the field, and they are worth restating because they define what success looks like. Database servers in that environment run RHEL or Ubuntu with four to five disks beyond the OS: /var, /log, /data and /backup, each on its own LUN, each with its own growth schedule. When a customer asks for more space on /data, somebody has to find out which hypervisor disk that actually is, grow the right one, then walk a chain of commands inside the guest to make the filesystem see the new space.

Why device letters lie

The core of the problem is that Linux device letters are an enumeration accident, not an identity. The kernel numbers SCSI devices in the order the bus scans them, and that order is a side effect of driver probing and device registration timing. It usually matches the configuration order, so /dev/sdb quietly becomes a false friend: the mapping appears stable right up until the day it is not, typically after a reboot, a hotplug event, or the addition of another disk. Nothing in the system promises that scsi3 is the third disk you configured.

The consequence in that environment was *resize-by-trial-and-error*: grow a disk by a recognizable amount on the hypervisor, watch which device letter changed size inside the guest, and only then know which disk you were actually touching. Multiply that discovery dance by ten servers in a batch and the error surface becomes serious: growing the wrong disk is harmless but wastes a maintenance window, while formatting the wrong disk is a data-loss incident. The fix is not more care; the fix is an identity that survives enumeration.

The manual resize ritual

Once the right disk was found, the ritual itself was five commands deep: `growpart` or `fdisk` to stretch the partition, `pvresize` to teach LVM about the new size, `lvextend` to grow the logical volume, and `xfs_growfs` to grow the filesystem. None of these steps is hard; all of them are forgettable, order-dependent, and easy to run against the wrong device when ten terminals are open at once. A whole-VM single-disk layout avoids the chain but recreates the I/O mixing problem, and converting a running ext4 system to LVM+XFS after the fact reportedly took some eighty commands to survive. Phase 2 deletes the ritual, not by scripting the five commands, but by making the question "which disk is this" unambiguous in the first place.

serial = identity

The disk letter is where the kernel found the disk. The serial is what the disk is. Address disks by what they are.

3. Design: The Serial Is the Disk's Identity

Every disk in this design carries a serial string set in Terraform, passed through the Proxmox configuration, presented by QEMU to the guest, and reported by the kernel in the SERIAL column of lsblk. The serial travels the whole stack unchanged, which is what makes it an identity rather than a label: it is the same value in the tfvars file, in the PVE configuration, and in the guest's kernel view of its own disks. The field run added one lesson to that chain, and the version below records it honestly.

THE IDENTITY CHAIN, END TO END (AS THE FIELD RUN VERIFIED IT)

```
terraform.tfvars data_disk_size_gb = 20
terraform (vm.tf) disk { interface = "scsi3" serial = "data01" size = 20 }
pve-a (qm config 200) scsi3: nfs-shared:vm-200-disk-2,size=20,serial=data01
pve-a (qm showcmd 200) serial=data01 on the scsi-hd device line
guest (lsblk -o SERIAL) sdc disk 20G data01 <- the kernel view
ansible (02_storage.yml) dev: "/dev/sdc" (resolved fresh from lsblk)
result vg_data/lv_data mounted at /srv/data
```

The field lesson sits in the last two hops. The first edition of this playbook resolved serials through /dev/disk/by-id, the classic udev path; on this stack that directory never materialized the serial-named entries. The kernel reported all four serials in lsblk while by-id offered none of them, which means Ubuntu 26.04 on virtio-scsi does not guarantee those symlinks. The playbook therefore reads the kernel view directly, and the device letters it resolves are safe to use precisely because nothing persists them: LVM tracks physical volumes by UUID, fstab mounts /dev/vg/lv device-mapper paths, and the mapping is rebuilt from scratch on every run. The letter is a handle for the duration of one playbook run; the serial, the VG and the LV are the identities that survive.

What the provider source guarantees

As in Phase 1, the design rests on provider source code, not on documentation folklore. Three properties were read directly from the bpg/proxmox v0.111.1 source before a single line was written, and they decide the whole shape of this phase. If any of them were false, the design would have to change.

Claim	Source evidence	Consequence for the design
Adding a disk to an existing VM does not force recreation	The disk block in disk.go contains no ForceNew flags	The four disks attach to the running VM 200 in-place; no destroy, no re-clone
Growing a disk is an online API call	vm.go builds resizeBodies and calls the resize API for size increases while the VM runs; only a datastore move sets shutdownForDisksRequired	The resize demo works without a reboot; a move between datastores is the one storage change that still needs downtime
Shrinking is refused, loudly	vm.go returns the error "Cannot shrink ... it is not supported"	Grow-only is enforced at the provider layer already; the playbook aligns with it instead of fighting it

Table 3. Design constraints verified in the provider source before implementation.

Layout decisions

On top of those guarantees, the layout follows five decisions. Each one is a trade-off that was considered against the alternatives, and the table records the reasoning so that future phases do not relitigate them by accident. The most consequential decision is one VG per disk: it keeps the resize chain unambiguous, because a grown disk grows exactly one VG and exactly one LV, with no free-space accounting between siblings.

Decision	Reasoning
One VG and one LV per disk	Disk grows, its VG grows, its LV grows: a one-to-one chain with no ambiguity. A shared VG across disks is more flexible for allocation but blurs which physical disk feeds which LV
LV sized at 100%VG	The LV always equals its disk; lvol extends it automatically when the VG grows, and idles as a no-op when nothing changed
XFS, not ext4	DB-server practice in the target environment; grows online with xfs_growfs; its inability to shrink matches the provider grow-only rule
Lab mounts under /srv/*	Mounting a fresh filesystem over a live /var needs a data migration first; the lab proves the machinery on safe mount points, production applies it with the rsync dance
Sizes are tfvars knobs	The golden template stays lean and identical for every VM; per-VM sizes are workload parameters, which is exactly what the Phase 4 request form will fill in

Table 4. Layout decisions and the reasoning behind each.

One property of this layout is worth internalizing early because it removes a whole class of anxiety: the design is grow-only, and that is a feature. The provider refuses to shrink a disk, XFS cannot shrink a filesystem, and the playbook runs `lvol` with `shrink: false` so it never tries. When a volume genuinely must become smaller, the honest procedure is a new disk of the right size plus a data move, which is a planned migration rather than an in-place illusion.

4. Terraform: Declaring the Four Disks

The Terraform side of Phase 2 is a single concept applied four times: a disk block with a serial. The excerpt below is the complete addition to `vm.tf`, sitting right after the existing `scsi0` OS disk block from Phase 1. Note what is absent: no provider-specific workarounds, no ignores, no lifecycle blocks. The identity scheme needs nothing beyond a string attribute that the provider passes through to the hypervisor.

TERRAFORM/VM.TF (PHASE 2 EXCERPT)

```
disk {
  datastore_id = var.storage_id
  interface = "scsi1"
  size = var.var_disk_size_gb
  serial = "var01"
  ssd = true
  discard = "on"
}

disk {
  datastore_id = var.storage_id
  interface = "scsi2"
  size = var.log_disk_size_gb
  serial = "log01"
  ssd = true
  discard = "on"
}

disk {
  datastore_id = var.storage_id
  interface = "scsi3"
  size = var.data_disk_size_gb
  serial = "data01"
  ssd = true
  discard = "on"
}

disk {
  datastore_id = var.storage_id
  interface = "scsi4"
  size = var.backup_disk_size_gb
  serial = "backup01"
  ssd = true
  discard = "on"
}
```

The `ssd` and `discard` flags mirror the OS disk so that all five disks behave the same way on the NFS backend: SSD emulation lets the guest issue TRIM, and `discard` on keeps thin provisioning honest on the share. The interfaces are pinned explicitly, `scsi1` through `scsi4`, because the interface is what the provider uses to match disks between state and reality when it computes changes; a stable pin keeps that diff clean.

The sizes come from four new variables, each with a default, which is what makes the change non-breaking for an existing `tfvars` file. The example file documents the knobs together with their serial, interface and mount point so that the mapping stays readable at a glance in code review.

TERRAFORM/TERRAFORM.TFVARS.EXAMPLE (PHASE 2 EXCERPT)

```
# --- Phase 2: data disks (one per mount point; grow = edit + apply) ---
var_disk_size_gb = 10 # scsi1 serial=var01 -> /srv/var
log_disk_size_gb = 10 # scsi2 serial=log01 -> /srv/log
data_disk_size_gb = 20 # scsi3 serial=data01 -> /srv/data (resize demo)
backup_disk_size_gb = 15 # scsi4 serial=backup01 -> /srv/backup
```

Variable	Default	Serial	Mount point
<code>var_disk_size_gb</code>	10	<code>var01</code>	<code>/srv/var</code>
<code>log_disk_size_gb</code>	10	<code>log01</code>	<code>/srv/log</code>
<code>data_disk_size_gb</code>	20 (demo grows to 30)	<code>data01</code>	<code>/srv/data</code>
<code>backup_disk_size_gb</code>	15	<code>backup01</code>	<code>/srv/backup</code>

Table 5. The four size knobs and where each one lands.

What the plan shows

With the repository updated, a plan against the existing Phase 1 state shows an update in place, not a replacement. This is the moment where the `no-ForceNew` proof from chapter 3 becomes visible in actual output, and it is worth reading carefully before applying: the plan must show exactly one resource changing and zero resources being destroyed.

EXPECTED PLAN (AGAINST THE LIVE PHASE 1 STATE)

```

~ resource "proxmox_virtual_environment_vm" "test_vm" {
~ agent { enabled = true }
~ disk {
~ serial = "data01" -> (known after apply)
# (5 unchanged attributes hidden)
}
+ disk {
+ datastore_id = "nfs-shared"
+ interface = "scsi1"
+ serial = "var01"
+ size = 10
}
+ disk { ... scsi2, serial "log01", size 10 ... }
+ disk { ... scsi3, serial "data01", size 20 ... }
+ disk { ... scsi4, serial "backup01", size 15 ... }
}

```

Plan: 0 to add, 1 to change, 0 to destroy.

The exact rendering of the diff varies with the provider version, but the summary line is the contract: 0 to add, 1 to change, 0 to destroy. If a plan ever shows the VM being replaced after a size edit, stop and read the diff before typing yes; in this design the only legitimate causes of replacement are changes to identity attributes like the VMID or the clone source, never a disk size.

5. Apply: Disks Appear While the VM Runs

Applying the disk addition is unremarkable by design: the provider updates the VM configuration through the API, creates the four disk files on the NFS datastore, and the running VM sees four new virtio-scsi devices through hotplug. There is no downtime window to schedule and no console to open; the whole operation typically takes seconds per disk on NFS.

EXPECTED APPLY (DISK ATTACH)

```

proxmox_virtual_environment_vm.test_vm: Modifying... [id=200]
proxmox_virtual_environment_vm.test_vm: Modifications complete after 12s

```

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

The one soft spot in this flow surfaced during the field run and is worth knowing before it bites. The API reports the disks attached and qm config shows the serials, but a serial is not a hotpluggable property: when the disks were attached after the VM had already started, the stored configuration carried the serials while the running QEMU process did not, so the guest kernel reported empty serials. The fix is a full stop and start on the node, `qm shutdown 200 && qm start 200`, because a reboot inside the guest does not restart QEMU. The playbook's assert in the next chapter catches exactly this case with a

two-branch diagnosis message, so the failure is readable rather than mysterious.

Verify from two sides

As everywhere in this series, a claim is only as good as its receipt, and the disks now exist on two sides of the hypervisor boundary. Three commands cover the whole chain, and all of them should agree on the same four serials before the Ansible step touches anything. The first two run on a PVE node and show what the hypervisor believes, in the stored configuration and in the actually running QEMU process; the last one runs inside the guest and shows what the kernel actually sees. This is the exact command set that isolated the field bug in minutes.

TWO-SIDED VERIFICATION (FIELD COMMANDS)

```
# on pve-a: the stored configuration carries the serials...
root@pve-a:~# qm config 200 | grep -E "scsi[1-4]:"
scsi1: nfs-shared:vm-200-disk-0,size=10,serial=var01,ssd=1,discard=on
scsi2: nfs-shared:vm-200-disk-1,size=10,serial=log01,ssd=1,discard=on
scsi3: nfs-shared:vm-200-disk-2,size=20,serial=data01,ssd=1,discard=on
scsi4: nfs-shared:vm-200-disk-3,size=15,serial=backup01,ssd=1,discard=on

# ...and so does the running QEMU process:
root@pve-a:~# qm showcmd 200 --pretty | grep -o "serial=[a-z0-9]*"
serial=var01
serial=log01
serial=data01
serial=backup01

# in the guest: the kernel reports every serial in its own view
ubuntu@vm-test-01:~$ lsblk -o NAME,SIZE,TYPE,SERIAL
NAME SIZE TYPE SERIAL
sda 10G disk drive-scsi0 <- OS disk: QEMU fallback id, no serial
sdb 15G disk backup01
sdc 20G disk data01
sdd 10G disk log01
sde 10G disk var01
```

Three details in that output reward attention. The OS disk shows drive-scsi0, QEMU's fallback identifier for a disk configured without a serial; it is harmless because nothing in this design ever addresses the OS disk by serial. The disk file numbering looks alarming on purpose: the OS disk is vm-200-disk-0.qcow2 while the four raw data disks took vm-200-disk-0 through vm-200-disk-3, so the allocator reused indexes across different file extensions; it looks like a bug, it is an implementation detail, and the disk count, sizes and serials were all correct. And do not expect /dev/disk/by-id to mirror the lsblk output: on this Ubuntu 26.04 guest it never created serial-named entries at all, which is exactly why the playbook reads the kernel view instead.

6. The Ansible Storage Playbook

Phase 1 ran on builtin modules alone; Phase 2 is where the pipeline grows its first real dependency. LVM and mount management live in community collections, and the repository now pins them in a requirements file so the control node is reproducible from the repo alone. The install is one command on ctrl-01, and it is the only new prerequisite this phase introduces.

ANSIBLE/REQUIREMENTS.YML + INSTALL

```
collections:
- name: community.general
source: https://galaxy.ansible.com
- name: ansible.posix
source: https://galaxy.ansible.com

# on ctrl-01:
ansible-galaxy collection install -r requirements.yml
```

The playbook itself is one screen of variables and one screen of tasks, and its shape follows the identity chain from chapter 3: resolve, then build, then mount. The variables declare the complete desired state for all four disks in one place, so adding a fifth disk to the lab later is a tfvars line, a vm.tf block and one entry in this list.

PLAYBOOKS/02_STORAGE.YML (THE DATA_DISKS CONTRACT)

```
vars:
data_disks:
- serial: var01 # scsi1, size: var_disk_size_gb
vg: vg_var
lv: lv_var
mount: /srv/var
- serial: log01 # scsi2, size: log_disk_size_gb
vg: vg_log
lv: lv_log
mount: /srv/log
- serial: data01 # scsi3, size: data_disk_size_gb
vg: vg_data
lv: lv_data
mount: /srv/data
- serial: backup01 # scsi4, size: backup_disk_size_gb
vg: vg_backup
lv: lv_backup
mount: /srv/backup
```

Resolution comes first because every later task addresses a disk through its resolved path, and the field run decided where that resolution reads from. The first edition walked /dev/disk/by-id with a find task; the Ubuntu 26.04 guest never materialized the serial symlinks there, so v2 asks the kernel directly: one

lsblk -J call captures NAME, TYPE and SERIAL for every block device, a set_fact builds the serial-to-disk map from that JSON, a debug task prints the mapping so the run log shows which device each serial became, and an assert fails loudly if a serial resolves to anything other than exactly one disk. That assert is the guard against both the stale-QEMU case from chapter 5 and the classic typo, and its failure message carries a two-branch diagnosis instead of a stack trace.

RESOLUTION: READ THE KERNEL VIEW, MAP, SHOW, ASSERT

```
- name: Read the kernel serial view of all disks
ansible.builtin.command:
cmd: lsblk -J -o NAME,TYPE,SERIAL
register: lsblk_serials
changed_when: false

- name: Map every serial to its disk(s) (kernel view)
ansible.builtin.set_fact:
serial_map: >-
{{ serial_map | default({}) | combine({
item: (lsblk_serials.stdout | from_json).blockdevices
| selectattr('type', 'equalto', 'disk')
| selectattr('serial', 'equalto', item)
| map(attribute='name') | list
}) }}
loop: "{{ data_disks | map(attribute='serial') | list }}"

- name: Every serial must resolve to exactly one disk
ansible.builtin.assert:
that: serial_map[item.serial] | default([]) | length == 1
fail_msg: >-
Expected exactly one disk with serial {{ item.serial }}, found
{{ serial_map[item.serial] | default([]) | length }}. Diagnose in
the guest with lsblk -o NAME,SIZE,SERIAL. (a) Empty serials there
mean the running QEMU predates the config: check qm config, then
fully stop and start the VM; a guest reboot does NOT restart
QEMU. (b) If lsblk shows the serial, compare against vm.tf.
loop: "{{ data_disks }}"
```

With the mapping established, the build chain is four tasks, and their module semantics were read from the community.general source rather than assumed. The filesystem module with `fstype: lvm` maps creation to `pvcreate` and growth to `pvresize`; the `lvg` module runs `pvcreate` itself if needed and creates the volume group; the `lvol` module with `size: 100%VG` computes the LV target as the full VG size, creates the LV on first run and issues `lvextend --resizefs` whenever the VG grew, which for XFS dispatches `xfs_growfs` online. The shrink flag is set false to match the grow-only design.

THE BUILD CHAIN (ONE ITERATION PER DISK)

```
- name: Create/resize physical volumes
community.general.filesystem:
  fstype: lvm
  dev: "{{ disk_map[item.serial] }}"
  resizefs: true # pvcreate first run, pvresize every run

- name: Create volume groups (one per disk)
community.general.lvg:
  vg: "{{ item.vg }}"
  pvs: "{{ disk_map[item.serial] }}"

- name: Create/extend logical volumes at 100 percent of their VG
community.general.lvol:
  vg: "{{ item.vg }}"
  lv: "{{ item.lv }}"
  size: 100%VG # LV always equals its VG
  resizefs: true # lvextend --resizefs -> xfs_growfs
  shrink: false # grow-only, by design

- name: Create XFS filesystems
community.general.filesystem:
  fstype: xfs
  dev: "/dev/{{ item.vg }}/{{ item.lv }}"

- name: Mount filesystems and persist in fstab
ansible.posix.mount:
  path: "{{ item.mount }}"
  src: "/dev/{{ item.vg }}/{{ item.lv }}"
  fstype: xfs
  opts: defaults,noatime
  state: mounted # mount now AND write fstab
```

Two details in that chain reward attention. The mount task uses state `mounted`, which both mounts immediately and writes the `fstab` entry, so the playbook is not finished until the layout survives a reboot; and the whole disk is handed to LVM as a PV without a partition table, which is the modern single-disk-per-LV pattern and removes the `growpart` step from the `resize` chain entirely. There is no partition to stretch because there is no partition.

Finally, the `pre_tasks` block installs `lvm2` and `xfsprogs` before anything else runs, because Ubuntu cloud images ship with neither. This is the same class of gotcha as the missing guest agent in Phase 1: cloud images are deliberately minimal, and anything the workload needs beyond the base system becomes the pipeline's job to install. Forgetting this step produces a confusing early failure where the filesystem module cannot find `pvcreate`; the `pre_task` makes the dependency explicit and idempotent.

7. First Run and Verification

The first run is the only run that changes everything: after it, every task in the playbook is either a no-op, a cosmetic pass or a growth step. Run it from the ansible directory as in Phase 1. On the field run the recap showed five changed tasks, which is what building four complete filesystem stacks from bare disks looks like; the tooling pre-task landed on ok because an earlier aborted attempt had already installed lvm2 and xfsprogs, and on a pristine VM that task changes too.

FIRST RUN (FIELD TRANSCRIPT)

```
root@ctrl-01:~/proxmox-automation/ansible# ansible-playbook playbooks/02_storage.yml

PLAY [Storage - LVM and XFS on serial-tagged data disks] *****

TASK [Show the serial to device mapping (the guessing game this replaces)] ***
ok: [vm-test-01] => (item=var01) => {
  "msg": "var01 (vg_var/lv_var -> /srv/var) = /dev/sde"
}
ok: [vm-test-01] => (item=log01) => { "msg": "log01 (vg_log/lv_log -> /srv/log) = /dev/sdd" }
ok: [vm-test-01] => (item=data01) => { "msg": "data01 (vg_data/lv_data -> /srv/data) = /dev/sdc" }
ok: [vm-test-01] => (item=backup01) => { "msg": "backup01 (vg_backup/lv_backup -> /srv/backup) = /dev/sdb" }

TASK [Every serial must resolve to exactly one disk] *****
ok: [vm-test-01] => (item=var01) => { "msg": "All assertions passed" }
... (3 more, all passed) ...

TASK [Create/resize physical volumes] *****
changed: [vm-test-01] => (item=var01) ... (3 more)

TASK [Create volume groups (one per disk)] *****
changed: [vm-test-01] => (item=vg_var) ... (3 more)

TASK [Create/extend logical volumes at 100 percent of their VG] ***
changed: [vm-test-01] => (item=vg_var/lv_var) ... (3 more)

TASK [Create XFS filesystems] *****
changed: [vm-test-01] => (item=/dev/vg_var/lv_var) ... (3 more)

TASK [Mount filesystems and persist in fstab] *****
changed: [vm-test-01] => (item=/srv/var) ... (3 more)

PLAY RECAP *****
vm-test-01 : ok=14 changed=5 unreachable=0 failed=0
```

Read the mapping block first, before anything else: those four debug lines are the guessing game from chapter 2 being solved in plain text. Each serial, its VG/LV pair, its mount point and its resolved device sit on one line, and that line is what the whole design buys you. If any serial had failed to resolve, the

assert would have stopped the play right there with instructions instead of a stack trace.

verified in the field

This chapter and the next quote the actual lab transcripts: Ubuntu 26.04 LTS guest on kernel 7.0.0-30-generic, PVE 9.2, provider 0.111.1. All four serials resolved, failed=0 on every run of the session, and the device letters stayed stable across the whole day.

The playbook finishes with two receipts so that the run log ends with evidence rather than a claim. lsblk with the SERIAL column shows the full stack from device letter through serial to mount point in one table, and df shows the four filesystems mounted with their sizes. The shape after the first field run, with the device letters being whatever the kernel assigned that day:

RECEIPTS (FIELD SHAPE)

```
TASK [Show block devices with serials and mounts] *****
NAME SIZE TYPE SERIAL MOUNTPOINT
sda 10G disk / <- OS disk (no serial needed)
sdb 15G disk backup01
└─vg_backup-lv_backup 15G lvm /srv/backup
sdc 20G disk data01
└─vg_data-lv_data 20G lvm /srv/data
sdd 10G disk log01
└─vg_log-lv_log 10G lvm /srv/log
sde 10G disk var01
└─vg_var-lv_var 10G lvm /srv/var

TASK [Show filesystem usage] *****
Filesystem Size Used Avail Use% Mounted on
/dev/mapper/vg_var-lv_var 10G 104M 10G 2% /srv/var
/dev/mapper/vg_log-lv_log 10G 104M 10G 2% /srv/log
/dev/mapper/vg_data-lv_data 20G 424M 20G 3% /srv/data
/dev/mapper/vg_backup-lv_backup 15G 145M 15G 1% /srv/backup
```

Idempotence: run it again

The contract of this playbook is that a second run, with nothing changed anywhere, is boring, and the field run put precise numbers on that promise. The PVs already exist, the VGs already contain them, the LVs already equal their VGs, the filesystems already exist and the mounts already match fstab, so every task lands on ok except one: the physical volume task reported changed again on the field run, because the community.general.filesystem module with resizefs: true runs pvresize on every invocation and flags the task changed even when no size moved. The effect is still idempotent, nothing grows and nothing is rewritten, but the honest expectation for the recap is one changed task, not zero.

SECOND RUN (FIELD TRANSCRIPT)

```
root@ctrl-01:~/proxmox-automation/ansible# ansible-playbook playbooks/02_storage.yml

TASK [Create/resize physical volumes] *****
changed: [vm-test-01] => (item=var01)
changed: [vm-test-01] => (item=log01)
changed: [vm-test-01] => (item=data01)
changed: [vm-test-01] => (item=backup01) # the pvresize pass

# every other task: ok

PLAY RECAP *****
vm-test-01 : ok=14 changed=1 unreachable=0 failed=0
```

second run: changed drops to one

The one surviving change is the pvresize pass: with resizefs: true the module runs it unconditionally and reports changed even when no disk grew. Treat it as noise; the pass signal is that no VG, LV, filesystem or mount task reports a change.

8. The Resize Demo: One Edit, No Reboot

This chapter is the reason Phase 2 exists, and the field session executed it end to end on the live lab. The scenario: /srv/data is heading toward full on a server that must not go down. In the old world this meant a maintenance window, a trial-and-error search for the right hypervisor disk, and the five-command ritual. Here it is one variable edit, one apply and one playbook re-run, and the VM never notices. The transcripts below are the real ones, not mock-ups.

STEP 1 + 2: EDIT THE KNOB, APPLY ONLINE (FIELD TRANSCRIPT)

```
# on ctrl-01: data_disk_size_gb 20 -> 30, then apply
root@ctrl-01:~/proxmox-automation/terraform# terraform apply

~ resource "proxmox_virtual_environment_vm" "test_vm" {
~ disk {
~ size = 20 -> 30 # only scsi3 changes
}
}

Plan: 0 to add, 2 to change, 0 to destroy. # 2nd change: the template VM
... # refreshing computed attrs
proxmox_virtual_environment_vm.test_vm: Modifications complete after 1s

Apply complete! Resources: 0 added, 2 changed, 0 destroyed.
```

The apply resized the disk through the API while the VM was running; the virtio-scsi layer in the guest picked up the new device size on the spot. Nothing inside the guest changed yet, though: the PV still records the old size, the LV and the filesystem with it. The disk is bigger; the stack on top of it does not know. That is exactly what the playbook re-run is for.

order matters (learned the honest way)

The field session once ran the playbook before the apply, and df still reported 20G: the playbook can only converge what the hypervisor has already grown. The sequence is edit, terraform apply, playbook, receipt.

STEP 3 + 4: CONVERGE THE STACK, TAKE THE RECEIPT (FIELD TRANSCRIPT)

```
root@ctrl-01:~/proxmox-automation/ansible# ansible-playbook playbooks/02_storage.yml

TASK [Create/resize physical volumes] *****
changed: [vm-test-01] => (item=data01) # pvresize saw the growth
# (+ the usual pass on the rest)

TASK [Create/extend logical volumes at 100 percent of their VG] ***
changed: [vm-test-01] => (item=vg_data/lv_data) # lvextend --resizefs

PLAY RECAP *****
vm-test-01 : ok=14 changed=2 unreachable=0 failed=0

root@ctrl-01:~# ssh ubuntu@192.168.122.50 'df -h /srv/data'
Filesystem Size Used Avail Use% Mounted on
/dev/mapper/vg_data-lv_data 30G 620M 30G 3% /srv/data
```

Two tasks changed, and only one of them means growth: the LV task changed for exactly the one disk that grew, because lvol, seeing the VG larger than the LV, issued lvextend with --resizefs, which

dispatched `xfs_growfs` online while the filesystem stayed mounted. The PV task flagged its usual every-run pass on all four disks. That precision is the payoff of the one-VG-per-disk layout: growth is routed to exactly the right stack by the serial, with nothing adjacent touched. Uptime never changed during the whole sequence; the disk grew from 20G to 30G under a mounted, in-use XFS filesystem.

It is worth rehearsing the failure mode too, because someone will eventually try to shrink a disk this way. Setting `data_disk_size_gb` back to 20 makes the plan fail with the provider's own words: Cannot shrink, it is not supported. That error is a design feature doing its job. The honest path to a smaller volume is a new disk at the desired size, a data move, and removal of the old one, which is a planned migration with its own runbook, not an in-place trick.

9. Troubleshooting

The failure modes of this phase are few, and most of them fail early with a readable message thanks to the assert-first structure of the playbook. The table below covers every failure observed in the field run or anticipated during design, ordered roughly by when in the flow they can occur. As in Phase 1, the column on the right is the actual fix, not a description of the problem restated.

Symptom	Cause	Fix
Plan shows the VM being destroyed and recreated	An identity attribute changed (VMID, clone source), not a disk size	Stop before yes and read the diff; restore the identity attribute, never accept a replace for a size edit
Assert fails: serial resolves to 0 disks, lsblk shows empty serials	The running QEMU predates the config: serials are not hotpluggable into a running process	qm config shows the serials, so fully stop and start the VM on the node (qm shutdown 200 then qm start 200); a guest-OS reboot does NOT restart QEMU
Serials visible in lsblk but /dev/disk/by-id has no serial entries	Observed in the field: this Ubuntu 26.04 virtio-scsi guest never creates the by-id symlinks	Nothing to fix: playbook v2 reads the kernel view; by-id was the first edition's dependency and is gone
df still shows the old size after editing a size knob	The playbook ran before terraform apply grew the disk	Order matters: edit, terraform apply, then the playbook; it converges only what the hypervisor already grew
Module lvol or lvg not found	Collections not installed on the control node	ansible-galaxy collection install -r requirements.yml
Filesystem module fails looking for pvcreate	Cloud image without lvm2 (the pre_task was skipped or removed)	Re-run the playbook unmodified; the pre_task installs lvm2 and xfsprogs first
Apply fails during disk creation	NFS datastore out of space	Free space on the share or lower a size knob; partially created disks are cleaned up by the provider retry
Cannot shrink error on plan or apply	A size knob was lowered	Grow-only by design; if the volume must be smaller, create a new disk and migrate
DEPRECATION WARNING lines under the mount task	The community.general mount module still imports old ansible-core APIs	Harmless collection noise, not a repo defect; ignore it, or set deprecation_warnings = False in ansible.cfg
Device letters changed after a reboot	Normal kernel enumeration behavior	Nothing to fix: the playbook addresses disks by serial; LVM tracks PVs by UUID and fstab mounts /dev/vg/lv, letters are cosmetic

Table 6. Phase 2 failure modes and their actual fixes.

One entry in that table deserves a closing comment because it is the philosophy of the whole phase: device letters changing after a reboot is listed as a symptom people notice, but in this design it is genuinely nothing. The letters were never load-bearing. The serials were, and the serials cannot shuffle, because they are part of the disk configuration stored on the hypervisor side, not an accident of the boot

scan.

10. Where the Series Goes Next

With Phase 2 closed, a freshly provisioned VM arrives with its OS, its identity, its networking and its complete DB-grade storage layout, all from one apply and one playbook. What it still lacks is everything that made the original manual process slow after the storage step: the agents. Phase 3 turns the old template's `/install` directory into proper Ansible roles, one per agent, each idempotent and each config-driven: Zabbix Agent 2 with its server address templated into `zabbix_agent2.conf`, Trend Micro and ManageEngine installed from controlled sources, and rsyslog forwarding rules pointing at QRadar.

Phase 4 then removes the last manual touchpoint: the request form. The sizes introduced in this guide are already plain tfvars knobs, which means they are form fields waiting to happen; a Semaphore UI pipeline can write a workspace tfvars from a form submission, run terraform apply and both playbooks, and deliver a fully built server minutes later. Phase 5 hardens the pipeline for scale: IPAM so IP addresses stop being hand-assigned, parallelism limits, and drift detection to notice when reality wanders from the declared state.

The distance traveled is worth a paragraph of its own, because it is the metric that justifies the whole series. The process this series replaced ran twenty to ninety times a day: console-driven cloning, manual IP configuration, one SSH session per agent, hand-edited configs and a reboot to finish, with storage work layered on top whenever a DB profile was involved. After two phases, the same result is two commands from a control container, every step logged, every failure mode documented, and every disk addressed by what it is rather than by where the kernel happened to find it. The remaining phases are repetition of the pattern, not new ideas: declare the state, converge onto it, verify with receipts.